

sample-workflow

Sample Workflow: User Authentication Feature

This example walks through a complete superspec workflow for a “User Authentication” feature in a web application. It demonstrates each command, shows abbreviated outputs, and illustrates superpowers integration with fallback behavior.

Setup

Assume we have a fresh project with no `.specify/` directory yet.

Phase 0: Create Constitution

User: `/speckit.constitution TaskFlow`

The agent interviews us about project principles, tech stack, and quality gates. After the conversation, it creates:

`.specify/memory/constitution.md` (abbreviated):

```
# TaskFlow Constitution
```

```
## Core Principles
```

```
### I. Simplicity First
```

```
Start simple, YAGNI. Only add complexity when current design  
cannot solve the problem.
```

```
### II. Test-First
```

```
TDD mandatory for all business logic. RED-GREEN-REFACTOR strictly  
enforced.
```

```
### III. Security by Default
```

```
Authentication and authorization on every endpoint. Input  
validation at boundaries.
```

Technology Stack

Layer	Technology	Purpose
Frontend	React + TypeScript	SPA
Backend	Express + TypeScript	REST API
Database	PostgreSQL	Primary storage
Testing	Vitest + Supertest	Unit + integration

Development Workflow

(superspec pipeline: constitution → specify → brainstorm → plan → tasks → execute → review)

Quality Gates

- Unit tests: REQUIRED (80% coverage)
 - TDD discipline: REQUIRED
 - Code review: REQUIRED for API contracts and security
-

Phase 1: Write Feature Spec

User: /speckit.specify "User authentication with email/password login, registration, and password reset"

The agent reads the constitution, interviews us about user stories, and creates:

specs/001-user-authentication/spec.md (abbreviated):

Feature Specification: User Authentication

****Feature Branch**:** ``001-user-authentication``

****Created**:** 2026-04-22

****Status**:** Draft

User Scenarios & Testing

User Story 1 - Email Registration (Priority: P1)

As a new user, I want to register with my email and password so I can access the app.

****Acceptance Scenarios**:**

1. Given a valid email and password, When I submit registration, Then my account is created
2. Given an already-registered email, When I submit, Then I see "Email already in use"

User Story 2 - Login (Priority: P1)

As a registered user, I want to log in with my email and password.

User Story 3 - Password Reset (Priority: P2)

As a user who forgot my password, I want to reset it via email.

Edge Cases

- What happens with extremely long passwords?
- How do we handle concurrent registration with the same email?

Brainstorm Prompts

(pre-populated prompts for edge case exploration)

Open Questions

#	Question	Status	Resolution
Q1	Maximum password length?	Open	–

Requirements

- FR-001: System MUST validate email format
 - FR-002: System MUST hash passwords with bcrypt
 - FR-003: System MUST issue JWT tokens on successful login
 - ...
-

Phase 2: Brainstorm Edge Cases

User: /speckit.superspec.brainstorm specs/001-user-authentication/spec.md

The agent checks for superpowers:

Checking for brainstorming skill...

✓ Found brainstorming at ~/.agents/skills/brainstorming/SKILL.md
Using enhanced questioning protocol.

(If not found, the agent would say: "Superpowers brainstorming not detected, using built-in questioning protocol.")

The agent then asks questions one at a time:

Agent: Let's explore boundary conditions first.

What should be the minimum and maximum password length?

- A) 8-64 characters (common standard)
- B) 8-128 characters (generous)
- C) 12-256 characters (strict minimum, very generous max)
- D) Other

User: A) 8-64 characters

Agent: Good. Now for error scenarios:

What happens if a user tries to register and the database is temporarily unavailable?

- A) Show a generic "try again later" error

- B) Queue the registration and process when DB recovers
- C) Show specific error with retry button

User: A) Generic error with retry suggestion

Agent: Security question: Should we implement rate limiting on login attempts?

- A) Yes, lock after 5 failed attempts for 15 minutes
- B) Yes, progressive delay (1s, 2s, 4s, 8s...)
- C) Yes, CAPTCHA after 3 failed attempts
- D) No rate limiting needed

User: C) CAPTCHA after 3 attempts

After the session, the spec is updated:

Open Questions

#	Question	Status	Resolution
Q1	Maximum password length?	Resolved	8-64 characters
Q2	DB unavailability handling?	Resolved	Generic error with retry
Q3	Login rate limiting?	Resolved	CAPTCHA after 3 failed attempts

Brainstorm Log

Session 2026-04-22

****Focus****: Full edge case exploration

****Key insights****:

- Added password length limits (8-64 chars) to FR-002
- Added rate limiting requirement: CAPTCHA after 3 failed login attempts (new FR-008)
- Clarified error handling for DB unavailability
- Identified need for email verification flow (deferred to Phase 2)

****Spec updates****: Added FR-008 (rate limiting), FR-009 (email verification), updated edge cases

Phase 3: Create Implementation Plan

User: /speckit.plan

The agent reads the spec, researches the codebase, and creates:

specs/001-user-authentication/plan.md (abbreviated):

Implementation Plan: User Authentication

Summary

Implement email/password authentication with registration, login, password reset, and rate limiting. JWT-based session management.

Technical Context

****Language/Version****: TypeScript 5.x

****Primary Dependencies****: Express, bcrypt, jsonwebtoken, nodemailer

****Storage****: PostgreSQL with Prisma ORM

****Testing****: Vitest + Supertest

Constitution Check

Principle	Status	Notes
-----	-----	-----
Simplicity First	PASS	Standard auth patterns, no over-engineering
Test-First	PASS	TDD required for all auth logic
Security by Default	PASS	bcrypt, JWT, rate limiting, input validation

Execution Strategy

TDD Requirements

- [x] Auth service (login, register, password reset logic)
- [x] JWT token generation and validation
- [x] Rate limiting logic

Parallel Execution Opportunities

- User model + Auth middleware can be built in parallel
- Frontend auth pages can be built in parallel with backend API

Human Checkpoints

1. After foundational setup – verify DB schema
2. After registration flow – verify end-to-end
3. After login flow – verify JWT handling
4. Before merge – security review

Phase 4: Decompose Tasks

User: /speckit.superspec.tasks

The agent checks for superpowers:

Checking for writing-plans skill...

x writing-plans not detected, using template-based decomposition.

Creates **specs/001-user-authentication/tasks.md** (abbreviated):

Tasks: User Authentication

Phase 1: Setup

- [] T001 Initialize Prisma with PostgreSQL connection
- [] T002 [P] Configure Vitest + Supertest

Phase 2: Foundational

- [] T003 [TDD] Create User model with Prisma schema
- [] T004 [P] [TDD] Create auth middleware (JWT validation)
- [] T005 [P] [REVIEW] Define API route structure for /api/auth/*

****Checkpoint**:** Foundation ready. Review API contracts before proceeding.

Phase 3: User Story 1 - Registration (P1) MVP

- [] T006 [TDD] [US1] Registration service (validate, hash, create user)
- [] T007 [US1] POST /api/auth/register endpoint
- [] T008 [TDD] [US1] Email uniqueness validation
- [] T009 [US1] Registration error handling

****Checkpoint**:** Registration flow works end-to-end.

Phase 4: User Story 2 - Login (P1)

- [] T010 [TDD] [US2] Login service (verify credentials, issue JWT)
- [] T011 [US2] POST /api/auth/login endpoint
- [] T012 [TDD] [US2] Rate limiting with CAPTCHA trigger
- [] T013 [US2] Token refresh endpoint

****Checkpoint**:** Login flow works end-to-end.

Phase 5: User Story 3 - Password Reset (P2)

- [] T014 [TDD] [SUBAGENT] [US3] Password reset token generation
- [] T015 [SUBAGENT] [US3] Reset email sending via nodemailer
- [] T016 [TDD] [US3] Password reset confirmation endpoint

Phase 6: Polish

- [] T017 [P] [SUBAGENT] API documentation
- [] T018 [REVIEW] Security hardening review
- [] T019 Run full test suite – all tests must pass

Phase 5: Execute Implementation

User: /speckit.superspec.execute

The agent walks through tasks phase by phase:

Agent: Starting Phase 1: Setup.

[Implements T001: Prisma setup]
[Implements T002: Vitest config]

Phase 1 complete.

- T001 ✓ Prisma configured with PostgreSQL
- T002 ✓ Vitest + Supertest configured

All setup verified. Proceed to Phase 2?

User: Yes, proceed.

Agent: Starting Phase 2: Foundational.

T003 is marked [TDD]. Following RED-GREEN-REFACTOR:

1. Writing test for User model... ✓
2. Running test... FAIL (expected – model doesn't exist yet) ✓
3. Implementing User model... ✓
4. Running test... PASS ✓

T005 is marked [REVIEW]. API routes defined:

POST /api/auth/register
POST /api/auth/login
POST /api/auth/reset-password
POST /api/auth/reset-confirm
POST /api/auth/refresh

Please review these API contracts before I proceed.

User: Looks good, proceed.

Agent: Phase 2 complete. Proceeding to Phase 3: Registration...

Phase 6: Code Review

User: /speckit.superspec.review

Agent: Reviewing implementation against spec...

Checking for requesting-code-review skill...

✓ Found requesting-code-review at ~/.agents/skills/requesting-code-review/SKILL.md

Using enhanced review protocol.

Review Findings

****Spec Compliance****: 8/8 acceptance scenarios verified ✓

****Issues Found**** (confidence >= 80):

1. ****Critical**** (confidence: 95): Rate limiting resets on server restart

- File: src/services/rate-limiter.ts:42
- The in-memory counter is lost on restart. Consider Redis-backed storage.

- Recommendation: Add Redis adapter for rate limit state

2. ****Important**** (confidence: 85): Password reset tokens have no expiration

- File: src/services/auth.service.ts:128
- FR-010 requires tokens to expire after 1 hour, but no TTL is set.

- Recommendation: Add `expiresAt` field to reset token generation

No other issues above confidence threshold.

Summary

This walkthrough demonstrated:

1. **Constitution** → established Simplicity, Test-First, Security principles
2. **Specify** → created 3 user stories with acceptance scenarios
3. **Brainstorm** → discovered rate limiting, password length limits, error handling (with superpowers enhancement)
4. **Plan** → designed technical approach with constitution compliance check
5. **Tasks** → decomposed into 19 tasks with [TDD], [REVIEW], [SUBAGENT] markers
6. **Execute** → implemented with TDD discipline and human checkpoints
7. **Review** → found 2 issues with confidence-based scoring (with superpowers enhancement)

The spec file was the central artifact — brainstorming refined it, planning verified compliance with it, tasks traced back to its user stories, and review verified implementation against its acceptance scenarios.